



Programación Orientada a Objetos 2

Pruebas Unitarias y de Interfaz en una Aplicación de Gestión de Tareas

Creado por:

Julio César Méndez Sandigo

Docente:

José Durán García

Managua, 09 de Junio 2026

Introducción

Las pruebas automatizadas son una parte importante dentro del desarrollo de aplicaciones móviles, ya que permiten verificar que el sistema funcione correctamente antes de ser entregado o publicado. En esta práctica se desarrolló una aplicación Android sencilla para gestionar tareas pendientes, en la cual el usuario puede registrar tareas, visualizarlas en una lista, marcarlas como completadas, eliminarlas y consultar la cantidad de tareas pendientes.

El desarrollo se realizó utilizando Jetpack Compose para la construcción de la interfaz gráfica y Kotlin para la lógica del sistema. Además, se implementaron pruebas unitarias con JUnit para validar la clase encargada de administrar las tareas, y pruebas de interfaz para comprobar que los componentes visuales respondan adecuadamente ante las acciones del usuario.

Esta actividad permite comprender la diferencia entre probar la lógica interna de una aplicación y probar la interacción del usuario con la pantalla. También permite reconocer la utilidad de las pruebas negativas, las cuales ayudan a identificar fallos cuando el resultado esperado no coincide con el resultado real del sistema.

Objetivos

Objetivo general

Implementar pruebas unitarias y pruebas de interfaz utilizando Jetpack Compose para validar la lógica de negocio y el comportamiento visual de una aplicación Android orientada a la gestión de tareas pendientes.

Objetivos específicos

- Crear una aplicación Android funcional para registrar y administrar tareas pendientes.
- Diseñar una clase de modelo denominada Tarea con los atributos solicitados.
- Implementar una clase administradora para agregar, eliminar, completar, obtener y contar tareas.
- Construir una interfaz gráfica con Jetpack Compose que permita interactuar con la lista de tareas.
- Crear pruebas unitarias para validar la lógica de negocio de la aplicación.
- Crear pruebas de interfaz para verificar el comportamiento visual e interactivo de la pantalla.
- Diseñar una prueba negativa que falle intencionalmente y explicar la causa del error.
- Aplicar mejoras adicionales como filtros, ordenamiento alfabético y porcentaje de tareas completadas.

Descripción del problema

Una empresa desea desarrollar una aplicación móvil para gestionar tareas pendientes. La aplicación debe permitir registrar nuevas tareas, marcarlas como completadas, eliminar tareas y visualizar el listado completo de tareas registradas. Antes de publicar la aplicación, se requiere implementar pruebas automatizadas que garanticen el correcto funcionamiento de la lógica de negocio y de la interfaz gráfica.

La solución propuesta consiste en una aplicación Android con una pantalla principal en la que el usuario puede ingresar el título de una tarea, agregarla a una lista, visualizar la cantidad de tareas pendientes, completar una tarea y eliminarla. Además, se incorporan funciones adicionales para filtrar tareas completadas o pendientes, ordenar la lista alfabéticamente y mostrar el porcentaje de tareas completadas.

El uso de pruebas automatizadas permite comprobar que las operaciones principales del sistema se ejecuten correctamente, evitando errores en acciones esenciales como agregar, eliminar o completar tareas.

Desarrollo de la aplicación

Modelo Tarea

Se creó una clase de modelo denominada Tarea, la cual representa cada elemento registrado dentro de la aplicación. Esta clase contiene los atributos solicitados: id, título, descripción y estado. El estado permite identificar si la tarea se encuentra pendiente o completada.

Clase GestorTareas

La lógica principal de la aplicación se implementó en una clase administradora llamada GestorTareas. Esta clase se encarga de gestionar internamente una lista de tareas y contiene los métodos necesarios para agregar nuevas tareas, eliminar tareas existentes, marcar tareas como completadas, obtener las tareas pendientes y contar cuántas tareas se encuentran pendientes.

Además de los métodos principales, se implementaron funciones complementarias para obtener tareas completadas, ordenar tareas alfabéticamente y calcular el porcentaje de tareas completadas. Estas funciones permiten cumplir con el reto adicional planteado en la práctica.

Interfaz con Jetpack Compose

La interfaz fue diseñada utilizando Jetpack Compose. La pantalla principal contiene un campo de entrada para escribir el título de una tarea, un botón para agregarla, un listado de tareas implementado mediante LazyColumn, un indicador de cantidad de tareas pendientes y botones para completar o eliminar tareas.

También se agregaron botones de filtro para mostrar todas las tareas, solo las pendientes o solo las completadas. Asimismo, se incorporó una opción para ordenar las tareas alfabéticamente y un indicador que muestra el porcentaje de tareas completadas.

Figura 1. Pantalla principal de la aplicación.

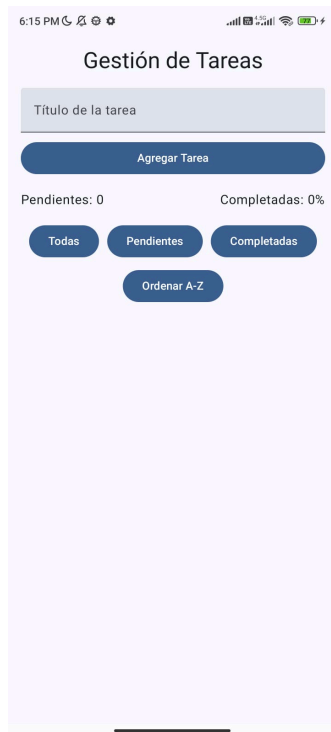


Figura 2. Registro de una nueva tarea.



Figura 3. Tarea marcada como completada.

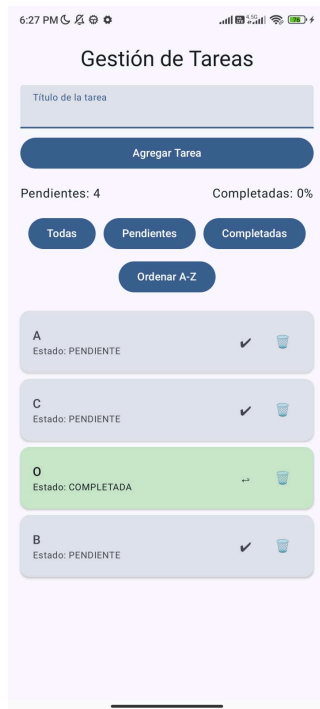


Figura 4. Eliminación de una tarea.



Requerimientos funcionales implementados

La aplicación desarrollada cumple con los requerimientos funcionales establecidos en la guía de la práctica. Cada requerimiento fue implementado dentro de la aplicación y validado mediante pruebas unitarias o pruebas de interfaz.

- Registrar nuevas tareas: el usuario puede escribir el título de una tarea y agregarla a la lista.
- Mostrar todas las tareas registradas: las tareas agregadas se visualizan en una lista dentro de la pantalla principal.
- Marcar tareas como completadas: cada tarea cuenta con un botón para cambiar su estado a completada.
- Eliminar tareas: cada tarea cuenta con un botón que permite eliminarla de la lista.
- Mostrar la cantidad total de tareas pendientes: la interfaz presenta un contador que indica cuántas tareas siguen pendientes.

Estos requerimientos representan las funciones principales de una aplicación de gestión de tareas y permiten comprobar que el sistema cumple con el objetivo planteado.

Pruebas unitarias

Las pruebas unitarias se implementaron utilizando JUnit y se ubicaron en la carpeta test del proyecto. Estas pruebas tienen como propósito validar la lógica de negocio sin necesidad de ejecutar la interfaz gráfica ni utilizar un emulador. En este caso, las pruebas unitarias verifican el comportamiento de la clase GestorTareas.

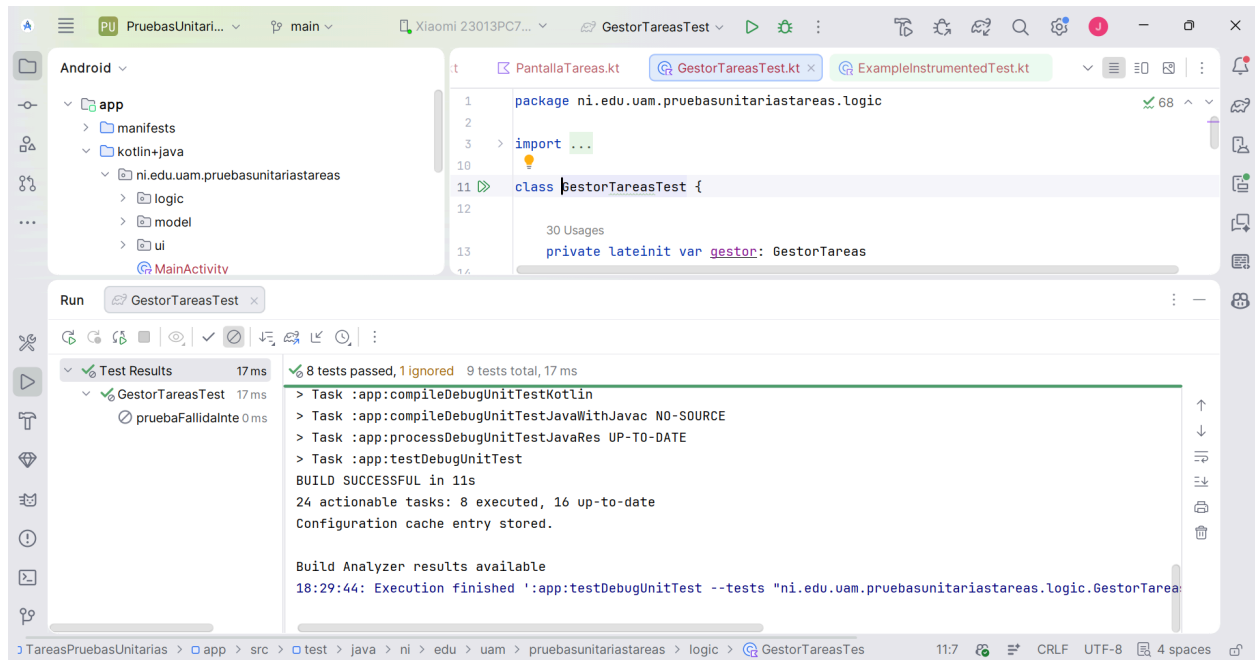
Los casos de prueba obligatorios fueron implementados para comprobar que al agregar una tarea la lista incremente en uno, que al eliminar una tarea esta desaparezca de la lista, que al completar una tarea su estado cambie correctamente, que el contador de tareas pendientes retorne el valor correcto y que una lista vacía retorne cero tareas pendientes.

Casos de prueba unitarios implementados

- Agregar una tarea: se verifica que la lista incremente su tamaño en uno.
- Eliminar una tarea: se verifica que la tarea eliminada ya no se encuentre en la lista.
- Completar una tarea: se verifica que el estado de la tarea cambie a completada.
- Contar tareas pendientes: se verifica que el método retorne la cantidad correcta de tareas pendientes.
- Lista vacía: se verifica que el contador de pendientes retorne cero cuando no existen tareas registradas.

Estas pruebas permiten comprobar que la lógica interna de la aplicación funciona correctamente y que las operaciones principales sobre la lista de tareas se ejecutan de acuerdo con lo esperado.

Figura 5. Pruebas unitarias ejecutadas correctamente.



Pruebas de interfaz con Jetpack Compose

Las pruebas de interfaz se implementaron en la carpeta androidTest del proyecto. A diferencia de las pruebas unitarias, estas pruebas requieren un emulador o un dispositivo Android, ya que interactúan directamente con los componentes visuales de la aplicación.

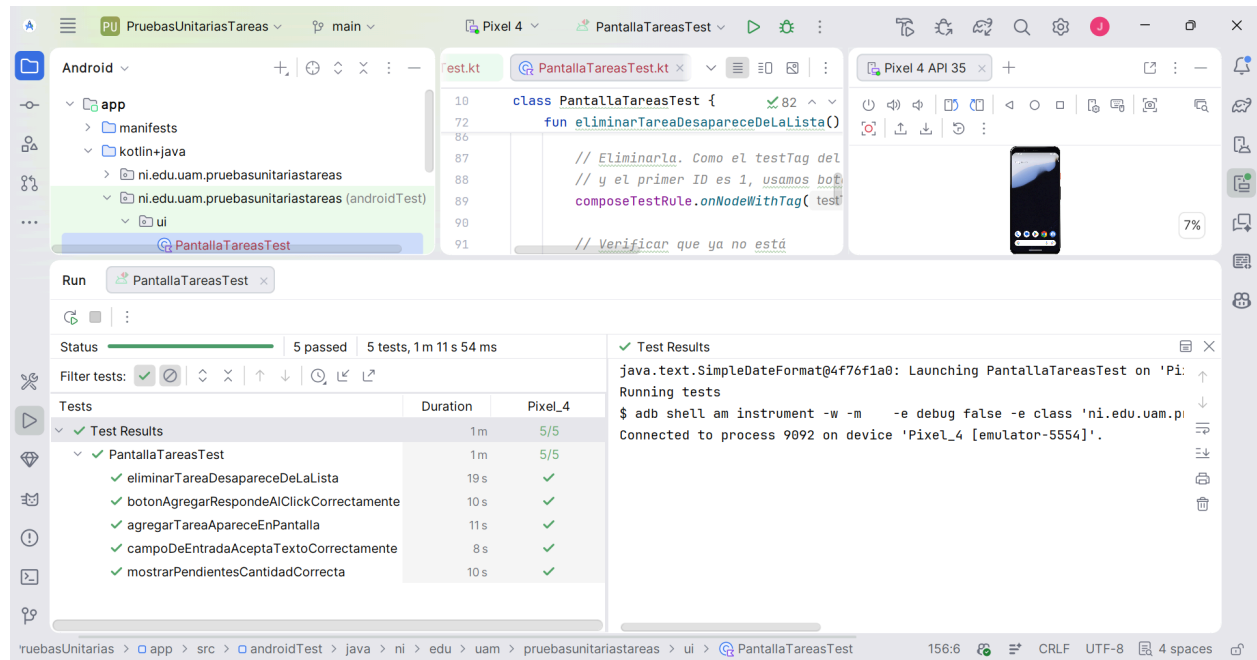
Estas pruebas validan que el campo de entrada acepte texto, que el botón Agregar responda correctamente al clic, que una tarea agregada aparezca en pantalla, que al eliminar una tarea esta desaparezca de la lista y que el contador de tareas pendientes muestre el valor correcto.

Casos de prueba de interfaz implementados

- Campo de entrada: se verifica que el TextField acepte el texto escrito por el usuario.
- Agregar tarea: se verifica que una tarea aparezca en la pantalla después de presionar el botón Agregar.
- Botón Agregar: se comprueba que el botón responda correctamente al clic.
- Eliminar tarea: se verifica que una tarea desaparezca de la lista después de presionar Eliminar.
- Mostrar pendientes: se verifica que el contador de tareas pendientes muestre la cantidad correcta.

El uso de pruebas de interfaz con Jetpack Compose facilita la localización de componentes mediante textos y etiquetas, permitiendo simular acciones reales del usuario y comprobar los resultados visibles en pantalla.

Figura 6. Pruebas de interfaz ejecutadas correctamente.



Prueba negativa

Como parte de la práctica se diseñó una prueba negativa que falla intencionalmente. La finalidad de esta prueba es demostrar cómo las pruebas automatizadas detectan errores cuando el resultado esperado no coincide con el resultado real devuelto por el sistema.

La prueba negativa consiste en agregar una tarea a la lista y luego esperar de forma incorrecta que el número de tareas pendientes sea cero. Sin embargo, al agregar una tarea, el resultado real del método que cuenta las tareas pendientes debe ser uno. Por esta razón, la prueba falla intencionalmente.

Explicación del fallo

La prueba falla porque el resultado esperado fue colocado incorrectamente. El valor esperado escrito en la prueba fue cero, pero el valor real devuelto por el sistema fue uno. Esto sucede porque la tarea agregada todavía se encuentra pendiente.

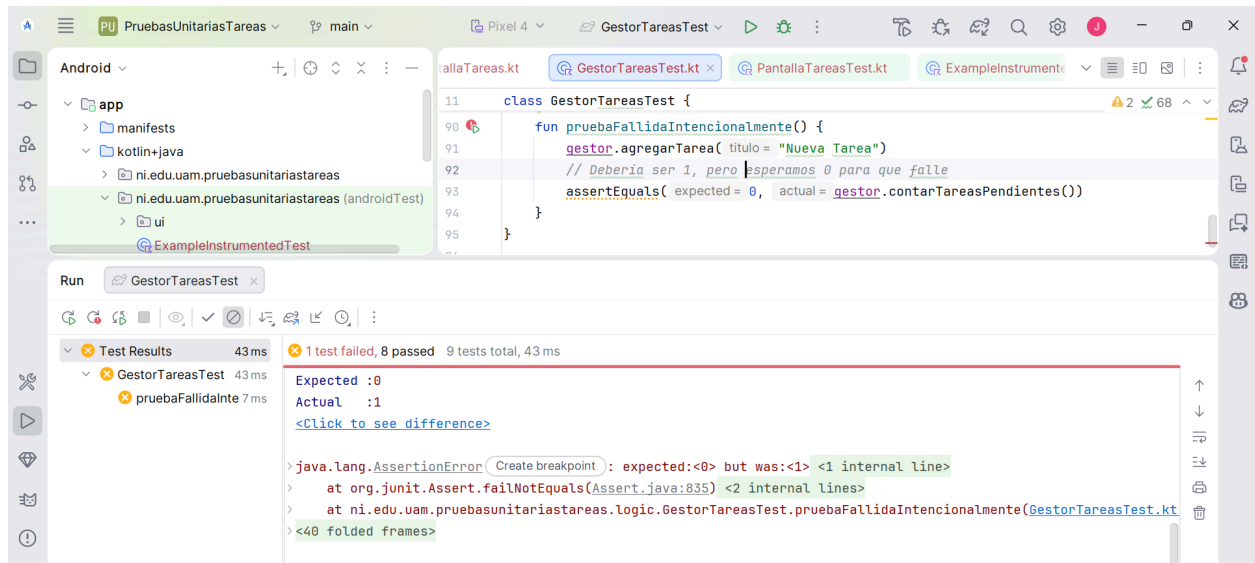
Resultado esperado

El resultado esperado correcto debería ser uno, ya que al registrar una nueva tarea esta se crea inicialmente con estado pendiente.

Corrección del error

Para corregir la prueba, se debe cambiar el valor esperado de cero a uno. De esta manera, la prueba validaría correctamente el comportamiento del sistema y pasaría sin errores.

Figura 7. Prueba negativa fallida intencionalmente.



Reto adicional

Además de los requerimientos principales, se implementaron mejoras adicionales solicitadas en la práctica. Estas mejoras permiten ampliar la funcionalidad de la aplicación y hacerla más útil para el usuario.

Filtro de tareas completadas

Se agregó una opción para mostrar únicamente las tareas que ya fueron marcadas como completadas. Esto permite al usuario revisar las tareas que ya han sido finalizadas.

Filtro de tareas pendientes

Se agregó una opción para mostrar únicamente las tareas que aún se encuentran pendientes. Esta función facilita identificar las tareas que todavía requieren atención.

Ordenamiento alfabético

Se incorporó una opción para ordenar las tareas alfabéticamente por título. Esto permite visualizar las tareas de una forma más organizada.

Porcentaje de tareas completadas

La aplicación muestra el porcentaje de tareas completadas con base en la cantidad total de tareas registradas. Este indicador permite conocer el avance general del usuario respecto a sus tareas.

Figura 8. Filtro de tareas pendientes.

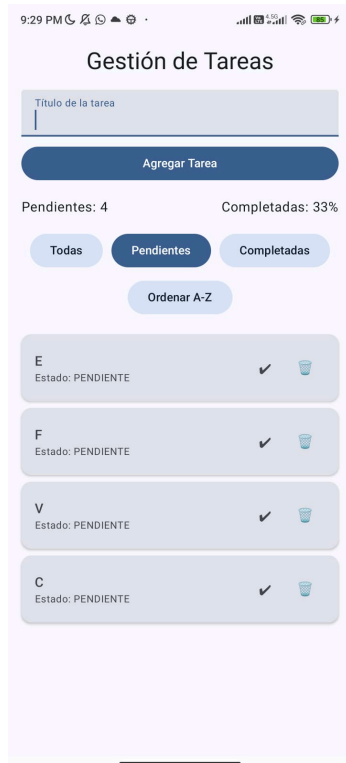


Figura 9. Filtro de tareas completadas.

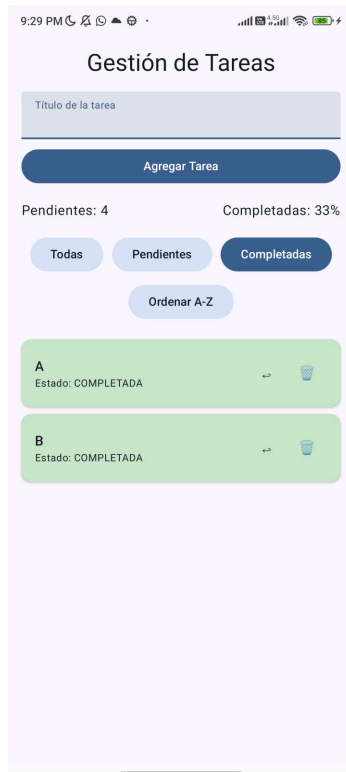


Figura 10. Ordenamiento alfabético de tareas.

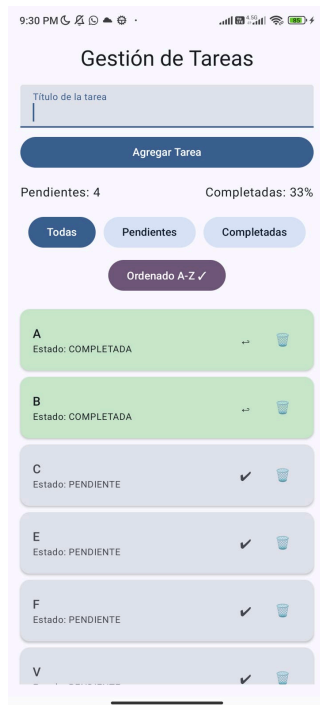
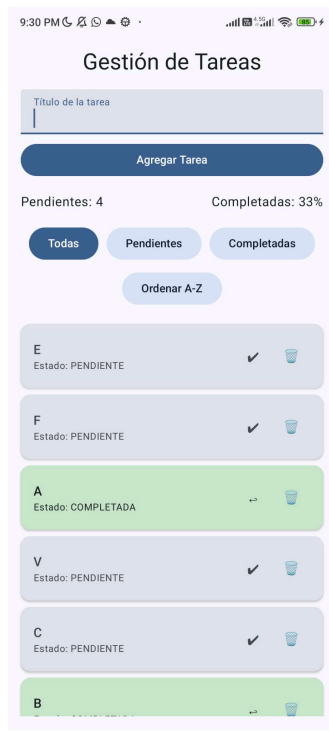


Figura 11. Porcentaje de tareas completadas.



Repositorio de GitHub

El código fuente completo del proyecto se encuentra disponible en un repositorio de GitHub. Este repositorio funciona como evidencia del código fuente completo de la aplicación, incluyendo los archivos principales, la clase de modelo Tarea, la clase GestorTareas, la interfaz creada con Jetpack Compose, las pruebas unitarias y las pruebas de interfaz.

Enlace del repositorio de GitHub:

https://github.com/JulioMS-DEV/Pruebas_Unitarias.git

Al incluir el repositorio, se cumple con el entregable relacionado con el código fuente completo del proyecto. Además, permite que el docente pueda revisar directamente la estructura del proyecto, los archivos implementados y las pruebas automatizadas desarrolladas.

Preguntas de análisis

1. ¿Por qué es importante implementar pruebas unitarias en esta aplicación?

Es importante implementar pruebas unitarias porque permiten comprobar que la lógica principal de la aplicación funciona correctamente sin necesidad de ejecutar toda la interfaz gráfica. En este caso, las pruebas unitarias validan operaciones como agregar, eliminar, completar y contar tareas pendientes. Esto ayuda a detectar errores rápidamente y facilita el mantenimiento del código.

2. ¿Qué parte de la aplicación se valida con las pruebas unitarias?

Las pruebas unitarias validan principalmente la lógica de negocio, representada por la clase GestorTareas. Esta clase administra la lista de tareas y contiene los métodos encargados de realizar las operaciones internas del sistema. Por esta razón, las pruebas unitarias no dependen de botones, pantallas ni componentes visuales.

3. ¿Qué parte de la aplicación se valida con las pruebas de interfaz?

Las pruebas de interfaz validan el comportamiento visual e interactivo de la aplicación. En esta práctica, se verifica que el campo de entrada acepte texto, que el botón Agregar funcione, que las tareas aparezcan en pantalla, que el botón Eliminar quite una tarea de la lista y que el contador de pendientes muestre la cantidad correcta.

4. ¿Cuál es la diferencia entre una prueba unitaria y una prueba de interfaz?

La prueba unitaria revisa una parte específica del código, como una clase o método, sin necesidad de abrir la aplicación. En cambio, la prueba de interfaz revisa el comportamiento de los componentes visuales cuando el usuario interactúa con ellos. Por ejemplo, una prueba

unitaria puede verificar que el método `contarTareasPendientes` retorne el valor correcto, mientras que una prueba de interfaz verifica que ese valor se muestre correctamente en pantalla.

5. ¿Para qué sirve una prueba negativa?

Una prueba negativa sirve para demostrar cómo el sistema de pruebas detecta un error cuando el resultado esperado no coincide con el resultado real. En esta práctica, la prueba negativa falla intencionalmente porque se espera que existan cero tareas pendientes después de agregar una tarea, aunque el resultado correcto es uno. Esto ayuda a comprender la importancia de definir correctamente los resultados esperados.

6. ¿Qué beneficios ofrece Jetpack Compose para realizar pruebas de interfaz?

Jetpack Compose facilita las pruebas de interfaz porque permite identificar elementos visuales mediante textos o etiquetas de prueba. Esto hace posible simular acciones del usuario, como escribir en un campo o presionar un botón, y luego verificar si la pantalla muestra el resultado esperado. Además, las pruebas pueden ejecutarse de forma automatizada en un emulador o dispositivo Android.

7. ¿Qué problemas pueden detectarse mediante pruebas automatizadas?

Las pruebas automatizadas pueden detectar errores en la lógica de negocio, fallos al actualizar la lista de tareas, problemas en el contador de pendientes, errores al eliminar o completar tareas y fallos en la interfaz gráfica. También ayudan a identificar si un cambio en el código afecta una funcionalidad que anteriormente funcionaba correctamente.

Conclusión

En esta práctica se desarrolló una aplicación Android para la gestión de tareas pendientes, utilizando Kotlin y Jetpack Compose. La aplicación permite registrar tareas, mostrarlas en una lista, marcarlas como completadas, eliminarlas y consultar la cantidad de tareas pendientes. Además, se incorporaron mejoras adicionales como filtros, ordenamiento alfabético y porcentaje de tareas completadas.

Las pruebas unitarias permitieron validar la lógica de negocio de la aplicación, comprobando que los métodos de la clase GestorTareas funcionaran correctamente. Por otro lado, las pruebas de interfaz permitieron verificar que los componentes visuales respondieran adecuadamente a las acciones del usuario, como escribir texto, presionar botones y visualizar resultados en pantalla.

La implementación de una prueba negativa permitió comprender cómo las pruebas automatizadas detectan errores cuando el resultado esperado no coincide con el resultado real. En conjunto, esta práctica demuestra la importancia de realizar pruebas antes de entregar o publicar una aplicación, ya que contribuyen a mejorar la calidad, confiabilidad y estabilidad del software.

Anexos

Repositorio de github:

https://github.com/JulioMS-DEV/Pruebas_Unitarias.git